

1. Why Operators Matter in Data Engineering

After reading and cleaning source data, a pipeline needs to make decisions.

For example:

```
source_count = 1500
target_count = 1475
file_received = True
schema_valid = True
error_message = None
```

We may need to determine:

- How many records were rejected?
- What percentage was rejected?
- Do source and target counts match?
- Is the file format supported?
- Is the schema valid?
- Is there any pipeline error?
- Should the pipeline finally pass or fail?

Operators are what allow Python to perform these calculations and validations.

2. Main Types of Operators

Operator Type	Purpose	Example
Arithmetic	Perform calculations	<code>source_count - target_count</code>
Comparison	Compare values	<code>source_count == target_count</code>
Logical	Combine conditions	<code>file_received and schema_valid</code>
Assignment	Assign/update values	<code>processed_count += 500</code>

Membership	Check presence	<code>".csv" in supported_formats</code>
Identity	Check objects / None	<code>error_message is None</code>

3. Arithmetic Operators

Important arithmetic operators:

- + Addition
- Subtraction
- * Multiplication
- / Division
- // Floor Division
- % Modulus
- ** Exponent

Rejected Record Calculation

```
source_count = 1500
target_count = 1475
```

```
rejected_count = source_count - target_count
```

```
print(rejected_count)
```

Output:

25

This means 25 source records did not reach the target.

Addition

Suppose two files are processed:

```
file_1_count = 800
file_2_count = 700
```

```
total_source_count = file_1_count + file_2_count
```

Result:

1500

Useful when combining file, batch or processed-record counts.

Division: Rejection Percentage

```
rejected_count = 25
```

```
source_count = 1500
```

```
rejection_percentage = (  
    rejected_count / source_count  
) * 100
```

```
print(f"{rejection_percentage:.2f}%")
```

Output:

1.67%

This percentage can later be compared against an allowed business threshold.

4. Floor Division and Modulus

Suppose:

```
total_records = 1500
```

```
batch_size = 200
```

Complete Batches

```
complete_batches = total_records // batch_size
```

Result:

7

Remaining Records

```
remaining_records = total_records % batch_size
```

Result:

100

So the processing can be understood as:

7 complete batches of 200

+

100 remaining records

This is useful when splitting records into chunks.

5. Comparison Operators

Comparison operators always return:

True

or

False

Important operators:

`==` Equal to

`!=` Not equal to

`>` Greater than

`<` Less than

`>=` Greater than or equal to

`<=` Less than or equal to

Source vs Target Validation

```
source_count = 1500
```

```
target_count = 1475
```

```
counts_match = source_count == target_count
```

Result:

False

Because the counts are different.

= VS ==

One of the most important beginner differences:

```
source_count = 1500
```

= assigns a value.

Whereas:

```
source_count == target_count
```

== compares two values.

Remember

= Assignment

== Comparison

6. Business Threshold Validation

Suppose the business allows a maximum rejection percentage of 5%.

```
rejection_percentage = 1.67
```

```
maximum_rejection_percentage = 5
```

```
within_limit = (  
    rejection_percentage  
    <= maximum_rejection_percentage  
)
```

Result:

True

Why <= Matters

If the rejection percentage is exactly 5:

5 <= 5

returns:

True

But:

5 < 5

returns:

False

So the operator must match the **business rule**.

7. Logical Operators

Logical operators combine multiple conditions.

and
or
not

and

and returns **True** only when **all required conditions are true**.

```
file_received = True  
schema_valid = True
```

```
validation_passed = (  
    file_received and schema_valid  
)
```

Result:

True

If:

```
schema_valid = False
```

the complete result becomes:

```
False
```

or

or is useful when **any one valid option is acceptable**.

Example: pipeline accepts either CSV or JSON.

```
file_name = "customers.csv"
```

```
is_supported_file = (  
    file_name.endswith(".csv")  
    or file_name.endswith(".json")  
)
```

Result:

```
True
```

not

not reverses a Boolean value.

```
file_received = True
```

```
file_missing = not file_received
```

Result:

```
False
```

8. Combining Multiple Pipeline Conditions

A production-style validation normally contains more than one check.

```
file_received = True
schema_valid = True
error_message = None
rejection_percentage = 1.67
maximum_rejection_percentage = 5

pipeline_successful = (
    file_received
    and schema_valid
    and error_message is None
    and rejection_percentage <= maximum_rejection_percentage
)
```

Result:

True

With **and**, every required condition must pass.

9. Assignment Operators

Assignment operators help update existing values.

```
=
+=
-=
*=
/=
```

Example:

```
processed_count = 0

processed_count += 500
processed_count += 400

print(processed_count)
```

Output:

900

This is equivalent to:

```
processed_count = processed_count + 500
```

Useful when processing multiple batches or files.

10. Membership Operators

Membership operators:

in
not in

They check whether a value exists inside a sequence or collection.

Suppose:

```
supported_formats = [  
    ".csv",  
    ".json",  
    ".parquet"  
]
```

Check:

```
file_extension = ".csv"
```

```
file_format_valid = (  
    file_extension in supported_formats  
)
```

Result:

True

Schema Validation Example

```
available_columns = [  
    "customer_id",  
    "customer_name",  
    "email"
```

```
]
```

Check whether `order_amount` is missing:

```
column_missing = (  
    "order_amount" not in available_columns  
)
```

Result:

True

11. Be Careful When Using `in` for File Extensions

This looks reasonable:

```
".csv" in "customers.csv.backup"
```

But it returns:

True

Even though the actual filename does not end in `.csv`.

Prefer:

```
file_name.endswith(".csv")
```

when validating an actual extension.

12. Identity Operators

Identity operators are:

`is`

`is not`

For this stage, their most important practical use is checking `None`.

```
error_message = None
```

```
if error_message is None:  
    print("No pipeline error")
```

To check whether an error exists:

```
if error_message is not None:  
    print("Pipeline error found")
```

Recommended

```
error_message is None
```

rather than:

```
error_message == None
```

13. == vs is

This is also interview-relevant.

```
value_1 = [1, 2, 3]  
value_2 = [1, 2, 3]
```

```
print(value_1 == value_2)  
print(value_1 is value_2)
```

Output:

```
True  
False
```

Why?

- **==** checks whether the **values are equal**.
- **is** checks whether both references point to the **same object**.

Practical Rule for Now

Use == for normal value comparison.

Use is None / is not None for None checks.

14. Operator Precedence

Python follows an order when multiple operators are present.

```
result = 10 + 5 * 2
```

Result:

20

Multiplication happens before addition.

Using parentheses:

```
result = (10 + 5) * 2
```

Result:

30

Pipeline Example

Prefer readable calculations:

```
rejection_percentage = (  
    rejected_count / source_count  
) * 100
```

Parentheses make the business logic easier to understand.

15. Real Pipeline Validation Example

Consider:

```
pipeline_name = "customer_daily_pipeline"
```

```
source_count = 1500
```

```
target_count = 1475
```

```
file_extension = ".csv"
```

```
file_received = True
```

```
schema_valid = True
```

```
error_message = None
```

```
maximum_rejection_percentage = 5
```

```
supported_formats = [  
    ".csv",  
    ".json",  
    ".parquet"  
]
```

Calculate rejected records

```
rejected_count = source_count - target_count
```

Result:

25

Calculate rejection percentage

```
rejection_percentage = (  
    rejected_count / source_count  
) * 100
```

Result:

1.67%

Perform validations

```
counts_match = source_count == target_count
```

```
file_format_valid = (  
    file_extension in supported_formats  
)
```

```
rejection_within_limit = (  
    rejection_percentage  
    <= maximum_rejection_percentage  
)
```

```
no_error = error_message is None
```

Final pipeline decision

```
pipeline_successful = (  
    file_received
```

```
and file_format_valid  
and schema_valid  
and rejection_within_limit  
and no_error  
)
```

Result:

True

16. Important Real-World Understanding

Notice:

Source Count = 1500
Target Count = 1475
Counts Match = False

But the pipeline can still be:

SUCCESS

Why?

Because the rejection percentage is only:

1.67%

and the allowed threshold is:

5%

Key Lesson

Source and target counts do not always need to match perfectly for a pipeline to succeed.

The success criteria depend on the business rule.

Operators simply implement that rule.

17. Pipeline Failure Scenario

Suppose:

```
target_count = 1350
schema_valid = False
error_message = "Required column email is missing"
```

Now multiple validations fail:

- Rejection percentage is too high.
- Schema is invalid.
- An error exists.

Because the conditions are combined using **and**, the final pipeline status becomes:

FAILED

18. Important Edge Case: Division by Zero

Suppose:

```
source_count = 0
```

This fails:

```
rejection_percentage = (
    rejected_count / source_count
) * 100
```

Python cannot divide by zero.

A basic check:

```
if source_count > 0:
    rejection_percentage = (
        rejected_count / source_count
    ) * 100
else:
    rejection_percentage = 0
```

An empty source should always be handled deliberately in a pipeline.

19. Common Mistakes

Mistake 1: Confusing `=` and `==`

Wrong comparison:

```
source_count = target_count
```

Correct:

```
source_count == target_count
```

Mistake 2: Using `and` Instead of `or`

Wrong:

```
file_name.endswith(".csv") and file_name.endswith(".json")
```

A file normally cannot end with both.

Correct:

```
file_name.endswith(".csv") or file_name.endswith(".json")
```

Mistake 3: Using `in` Carelessly

`".csv"` in `"customers.csv.backup"`

returns `True`.

Use:

```
file_name.endswith(".csv")
```

for extension validation.

Mistake 4: Ignoring Business Thresholds

A mismatch does not automatically mean failure.

For example:

Source = 1500

Target = 1499

Whether this should fail depends on the business requirement.

Mistake 5: Writing Complex Logic in One Line

Avoid difficult-to-read conditions.

Prefer:

```
pipeline_successful = (  
    file_received  
    and file_format_valid  
    and schema_valid  
    and rejection_percentage <= maximum_rejection_percentage  
    and error_message is None  
)
```

Readable pipeline logic is easier to debug and maintain.

20. Quick Revision

- `+` `-` `*` `/` perform calculations.
- `//` gives the complete whole-number division result.
- `%` gives the remainder.
- `==` compares values.
- `=` assigns values.
- Comparison operators return `True` or `False`.
- `and` requires all conditions to pass.
- `or` requires at least one condition to pass.
- `not` reverses a Boolean.
- `in` checks membership.
- `not in` checks absence.
- Prefer `is None` when checking `None`.
- `==` checks value equality; `is` checks object identity.

- Pipeline success should follow business rules and thresholds.
-

21. Student Practice Assignment

Validate the following pipeline:

```
pipeline_name = "daily_order_pipeline"
```

```
source_count = 2500
```

```
target_count = 2380
```

```
file_name = "orders_2026_07_31.json"
```

```
file_extension = ".json"
```

```
file_received = True
```

```
schema_valid = True
```

```
error_message = None
```

```
maximum_rejection_percentage = 4
```

```
supported_formats = [
```

```
    ".csv",
```

```
    ".json",
```

```
    ".parquet"
```

```
]
```

Calculate:

1. Rejected record count
2. Rejection percentage
3. Whether source and target counts match
4. Whether the file format is supported
5. Whether rejection percentage is within the limit
6. Whether there is no error
7. Final pipeline status

Expected:

Rejected Records: 120

Rejection Percentage: 4.80%

Pipeline Status: FAILED

Why?

Because:

4.80% > 4%

22. Practice Challenge

Change:

```
source_count = 2500
```

```
target_count = 2450
```

```
maximum_rejection_percentage = 4
```

Now calculate the rejection percentage again.

Expected:

2%

If every other validation passes, the pipeline should now succeed.

23. Interview-Relevant Questions

1. What is the difference between `=` and `==`?

`=` assigns a value to a variable.

`==` compares two values and returns `True` or `False`.

2. What do comparison operators return?

They return Boolean values:

`True`

`False`

3. What is the difference between **and** and **or**?

and requires every condition to be true.

or returns true when at least one condition is true.

4. What does **%** do?

The modulus operator returns the remainder after division.

Example:

1500 % 200

returns:

100

5. What is the difference between **==** and **is**?

== compares values.

is checks whether two references point to the same object.

For **None**, prefer:

value is None

6. Why are parentheses useful in pipeline conditions?

They make calculations and business rules easier to read and reduce mistakes in complex expressions.

7. If source and target counts do not match, should the pipeline always fail?

No.

The pipeline may still succeed if the business allows a rejection threshold and the actual rejection percentage remains within that limit.

8. Why should `source_count = 0` be handled separately?

Because calculating a rejection percentage would involve division by zero, which causes an error.

24. Mini Interview Challenge

Predict the outputs:

```
print(10 > 5 and 5 > 2)
print(10 > 5 and 5 < 2)
print(10 > 5 or 5 < 2)
print(not True)
print(None is None)
```

Expected:

```
True
False
True
False
True
```